SYS3 LAB ACTIVITY 3 : Power Efficiency

Week 10 (typical study period)

Introduction:

All electronic circuits consume power, and in addition they emit heat and radio waves as waste energy. By far the largest component of waste energy that we observe is of course heat - we all know that laptops and phones get hot when used.

In general terms, power consumption per mips has been reducing by about a factor of 2 every two to three years for a number of decades since silicon chips were invented (a trend known as Koomey's Law). In other words, the same computational effort requires half as much power every few years, around half for every 2.5 years in the current technology epoch - a little slower now than in the early years but still fairly consistent.

We typically think about optimising code (a) to remove hazards, and (b) to shorten execution time. However, taking the power consumption of a given CPU and a given piece of code, we could instead choose to organise the sequencing of instructions to reduce power consumption. Or we might see this in a different way: a poorly ordered sequence of instructions or a piece of code that is poorly optimised in other respects could consume a lot of power compared to the exact same code when fully optimised for pipeline hazards and 'intelligently' sequenced in a way that understands how to minimise power usage.

We learned in the previous week's work that instructions may be reordered under certain circumstances, so we already know that changing the order of instructions is possible. We also have solutions that eliminate pipeline hazards and allow code to execute in fewer cycles by having more functional units. But having more functional units will cost more in terms of power, even if it shortens the execution time. What then are the trade-offs between speed and power consumption ?.

An important further concern is that when we increase the complexity of a CPU, we potentially slow its clock rate down (i.e. the frequency at which instruction cycles can be completed). A large register file or large cache might cause this, as could the effect of having more functional units.

We can further divide power consumption in circuits, and particularly CPU's here, into static and dynamic power. **Static** power is consumed continuously, so a CPU

that does nothing for 20 clock cycles will still consume power at a given rate just like a car engine idling whilst waiting at a junction. Meanwhile **Dynamic** power occurs when parts of the CPU are actively doing work, for instance, functional units such as instruction decoder, ALU, register file, etc.

We could envisage that a piece of code where lots of avoidable delays occur will nonetheless use the functional units a given number of times, but it will also have a lot of static power. If the work can be done quicker, the dynamic power will not reduce, but the static power will. So we can already speculate that removing avoidable pipeline hazards and using the sequencing of instructions to fill idle periods with useful instructions should improve power efficiency.

This week's exercises focus on exploring that idea.

10.1 Initial Preparation

So far, in previous lab sessions, we have achieved a number of optimisation tasks, dealing with a range of hazards:-

1. Structural Hazards
2. Memory Hazards
3. Register Hazards

Solutions to all of these problems are combined in the test program Week10.cpp to give what appears to be a near-complete solution to the problem of hazards in pipelines using dynamic pipeline scheduling techniques.

This program can turn optimisation on and off using the -noopt command line option. For example:

./week10 testcases/test10a.txt -noopt

Try compiling the week10 test program and then running it as above. You will see that the test program outputs some extra data you have not seen in previous labs. It might look something like this:-

```
CACHE Mode 1 (SPLIT)
MEM   Static  1950.00, Dynamic    96.00, Total  2046.00
ID    Static   520.00, Dynamic   200.00, Total   720.00
IALU  Static   260.00, Dynamic   100.00, Total   360.00
FPALU Static   195.00, Dynamic     0.00, Total   195.00
SHALU Static   130.00, Dynamic     0.00, Total   130.00
RR    Static   260.00, Dynamic    60.00, Total   320.00
WB    Static    65.00, Dynamic    40.00, Total   105.00
TOTAL Static  4680.00, Dynamic   480.00, Total  5160.00
```

What we can see here is the static and dynamic power consumption for each kind of functional unit or structural resource in the CPU and how much power was consumed for the piece of code in test10a.txt.

This is imaginary power - it is based very loosely upon some assignments of power made in the pipeline toolset configuration. No units are given, but for individual clock cycles of individual resources in a real processor the power

consumed is likely to be very small, in the region of nano or pico joules and picowatts. When billions of these instructions are executed per second, the power can add up to tens of watts in a typically CPU.

We can see that the total static power in this example happens to be much higher than the dynamic power. This is because only a small fraction of the CPU resources are being used in any one clock cycle or over the whole program. It is rare for a CPU to be able to keep all functional units busy all of the time and as a result. the static power is burning into the power budget even when no useful work is being done.

We will now do some experiments to see how power changes with different design choices.

10.2 Experiment 1 - Cache & Memory

CPU Configuration

- Pipe.IssueWidth =1;
- Pipe.ReadPorts =4;
- Pipe.WritePorts =1;
- Pipe.IALUCount =2;
- Pipe.FPALUCount =1;
- Pipe.SHALUCount =1;
- Pipe.CacheMode =0;

Test Cases

- **test10b.txt**

Make sure the CPU is set as stated above in **week10.cpp**, and then compile and run the program with **test10b.txt** and view the output. We will see that the initial pipeline contained several hazards, including a memory hazard due to trying to do an instruction fetch and a data fetch in the same cycle.

We will see that the number of pipelined cycles is 12 and the total power consumption is TOTAL Static 3720.00, Dynamic 460.00, Total 4180.00.

We know from earlier weeks that using a split cache would solve the problem of IF+DS hazards, so now let's set **cachemode=1** in **week10.cpp** and repeat the test. You will now see that the pipelined cycles has reduced to 11 cycles - in other words the program has speeded up. However by choosing split cache we introduce higher power costs (the circuitry is more complex). and we see that the power data is as follows: TOTAL Static 3960.00, Dynamic 460.00, Total 4420.00

We can compare directly:

CYC=12 TOTAL Static 3720.00, Dynamic 460.00, Total 4180.00. (unified cache)

CYC=11 TOTAL Static 3960.00, Dynamic 460.00, Total 4420.00 (split cache)

We can see that the speedup is about 10% (12/11 = 1.09), but we have also increased power consumption by something like 6% (4420/4180 = 1.057).

Another way to look at this is that the power per clock cycle was 4180/12 = 343 in the unified cache case, but it is 4480/11 = 407 for split cache - nearly 20% increased power per clock cycle.

So we can conclude that the code example runs about 10% faster and consumes about 6% more power, but the average power per clock cycle increases by nearly 20%. We have traded off execution time against power.

We could just as easily argue the reverse, that removing a split cache might make power consumption better at the cost of slower program execution. So we have a choice of two optimisation goals and could choose either depending upon what we want.

10.2 Experiment 2 - Resource Scaling

CPU Configuration

- Pipe.IssueWidth =1;
- Pipe.ReadPorts =2;
- Pipe.WritePorts =1;
- Pipe.IALUCount =1;
- Pipe.FPALUCount =1;
- Pipe.SHALUCount =1;
- Pipe.CacheMode =0;

Test Cases

- **test10c.txt**

Run **week10.cpp** with the given configuration and test case **test10c.txt**. You will notice that there are a lot of IALU hazards due to there being not enough IALU resources. These hazards are fixed by inserting stalls, but one of the consequences of this is that the CPU static power will be consumed for a lot of clock cycles (16 cycles in this case). If we increase the number of IALU's we should see a reduction in clock cycles, and static power will reduce for most parts of the CPU. However we will also add new static power because two IALU units have twice as much static power as one, and so-on.

Complete the table overleaf and see what you can conclude about the number of IALU's needed in this case.

No of IALU	IALU Static	IALU Dyn.	Total Static	Total Dyn.	Total Power	Pipelined cycles
1	210	220	5670	860	6530	21
2	340	220	5100	860	5960	17
3	510	220	5610	860	6470	17
4	680	220	6120	860	6980	17

You should be able to see that above a certain number of IALU resources, the clock cycles do not improve, and in fact any additional ILAUs will just add power consumption with no benefit.

We will also see that total IALU dynamic power does not change - the same amount of useful work is still being done, but in less time. You will be able to see that total dynamic power divided by cycles increases , meaning more useful work is being done per clock cycle.

Meanwhile, overall power and particularly static power, changes as the program gets shorter or longer. Does it improve up to a point ?

What is the best choice of number of IALUs in this case ?

10.2 Experiment 3 - Renaming

CPU Configuration

- Pipe.IssueWidth =1;
- Pipe.ReadPorts =4;
- Pipe.WritePorts =2;
- Pipe.IALUCount =4;
- Pipe.FPALUCount =1;
- Pipe.SHALUCount =1;
- Pipe.CacheMode =0;

- Pipe.RegFileSize=1;

Test Cases

- **test10d.txt**
- **test10e.txt**

In this experiment we will look at the effect of register renaming. Configure the CPU as given above, and noting the new parameter **RegFileSize=1;**

The files test10d and test10e represent the same program without and then with register renaming. We might expect register renaming to improve performance.

Recompile and run **week10.cpp** with test case **test10d.txt** and **test10e.txt**. Note how many cycles and how much power is consumed in each case in the table below.

Then re-run **week10.cpp** on **test10e.txt** with **RegFileSize=2,** and add that data to the table too.

test case	**Cycles**	**Power**	**RegFileSize**
test10d	15	5975	1
test10e	14	5600	1
test10e	14	6110	2

The theory here is that by using register renaming performance should improve. We see that in terms of clock cycles reducing. So in terms of speed then there is a benefit. We also see that the power reduces as long as **RegFileSize=1**. So it seems like there is a clear benefit.

However, with **RegFileSize=2**, we see that the overall power cost has increased, albeit along with a faster execution time.

Why is this? The reason is that in order to support register renaming, we will typically need a larger register file since more register identities are in use in a given piece of code. But making the register file larger (twice the size in this case) has a cost too - power consumption increases.

Because the register file is twice as big, all of the costs associated with the register file are doubled, which includes the read ports, write ports and other power costs.

Reducing either or both of the read ports and write ports parameters might create a situation where the gains are much better both for clock cycles and power used. Try changing the read and write port numbers and repeat the test and see if you can find a case where the results are better without increasing the clock cycles for the two cases.